UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERIA

ESCUELA ACADEMICO PROFESIONAL DE INGENIERIA EN INFORMÁTICA Y SISTEMAS

CURSO DE PROGRAMACION AVANZADA

ARTICULO CIENTIFICO:

COMPARACION ENTRE ALGORITMO SHELLSORT Y QUICKSORT

ALUMNO: GIANCARLO RODRIGUEZ DEL CARPIO

CODIGO: 2018-119051

TACNA – PERÚ

2026

COMPARACION ENTRE ALGORITMO SHELLSORT Y QUICKSORT

COMPARISON BETWEEN SHELL SORT AND QUICK SORT ALGORITHM

Giancarlo Rodriguez Del Carpio

Facultad de Ingeniería - Escuela Académico Profesional de Ingeniería en Informática y Sistemas de la Universidad Nacional Jorge Basadre Grohmann

Tacna Perú, 2026

grodriguezc@unjbg.edu.pe

Resumen

El presente artículo trata de desarrollar la diferencia entre los algoritmos de ordenamiento shellsort y quicksort. Para ordenar 1,000, 10,000 y 100,000 datos se realizarán 50 pruebas aplicando los algoritmos de ordenamiento, el resultado permitirá concluir si estadísticamente sí existen diferencias significativas entre los tiempos de ordenación de los algoritmos ya mencionados.

Palabras clave: Algoritmos de ordenamiento, análisis comparativo entre los algoritmos de ordenamiento

Abstract

This article aims to explore the differences between the shellsort and quicksort sorting algorithms. Fifty tests will be conducted to sort 1,000, 10,000, and 100,000 data points, applying each algorithm. The results will determine whether there are statistically significant differences in sorting times between the two algorithms.

Keywords: Sorting algorithms, comparative analysis between sorting algorithms

1 Introducción

El ordenamiento de elementos o cosas, es un proceso que realiza con el fin de clasificar y ubicar en el menor tiempo las mismas. La ordenación es un proceso de reorganización de una lista de elementos en el orden correcto (Adhikari 2007; Buradagunta y col; 2020), de aquí que los algoritmos de ordenación son considerados como una parte importante de la gestión de datos (Adhikari 2007), en la informática y programación (Htwe Htwe 2019; Prajapati y col; 2017; Yang y col; 2011; Zhao y col; 2016) y en las ciencias computacionales (Buradagunta y col; 2020; Nasar 2019).

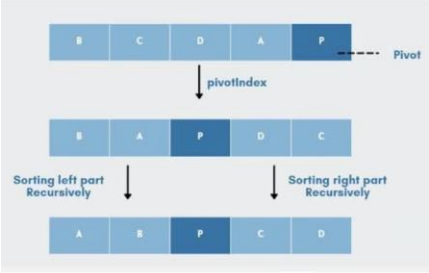
Un algoritmo de ordenación es un proceso computacional que facilita a los usuarios la clasificación de datos de forma rápida y automática considerando la velocidad en la clasificación de datos además de la eficiencia de la memoria (Iskandar y col; 2020).

La mayoría de los algoritmos de ordenación funcionan comparando los datos que se ordenan (Adhikari 2007) y cada algoritmo de ordenamiento es mejor en alguna situación, por ejemplo, el algoritmo de ordenamiento por inserción es preferible al de ordenación rápida cuando se tienen pocos elementos (Chowdhury 2016), además la implementación del algoritmo (matriz, vector, bases de datos, etc.) también será muy diferente, lo cual puede hacer que el mismo algoritmo que es rápido en un caso sea lento en el otro (Chowdhury 2016).

2 Metodología Experimental

ALGORITMO QUICKSORT

Quicksort es un algoritmo de ordenación rápido que funciona dividiendo una gran matriz de datos en submatrices más pequeñas. Esto implica que cada iteración divide la entrada en dos componentes, los ordena y luego los recombina. La técnica es altamente eficiente para grandes conjuntos de datos porque su complejidad promedio y en el mejor de los casos es O(n*logn).



Si bien Quicksort puede tener algunas desventajas, en la práctica funciona bien y supera a otros algoritmos de ordenación en velocidad y eficiencia espacial. Creado por Tony Hoare en 1961, Quicksort sigue siendo uno de los algoritmos de ordenación de propósito general más rápidos y efectivos disponibles en la actualidad

El algoritmo Quicksort funciona ordenando recursivamente las sublistas a ambos lados de un punto pivote dado y desplazando dinámicamente los elementos dentro de la lista alrededor de ese punto pivote.

Como resultado, el método Quicksort se puede resumir en tres pasos:

1. **Seleccionar:** Seleccione un elemento.
2. **Dividir:** Divide el conjunto de problemas, mueve las partes más pequeñas a la izquierda del punto de pivote y los elementos más grandes a la derecha.
3. **Repetir y combinar:** Repita los pasos y combine las matrices que se han ordenado previamente.

Veamos un ejemplo para comprender mejor el algoritmo Quicksort. En este ejemplo, el siguiente array contiene valores sin ordenar, que ordenaremos utilizando Quicksort.

ALGORITMO SHELLSORT

Shellsort es un algoritmo de ordenamiento basado en el enfoque de inserción, pero que mejora su eficiencia al permitir intercambios de elementos lejanos. Divide el arreglo en segmentos (determinados por un valor llamado "gap") y realiza múltiples pasadas de ordenamiento por inserción con valores decrecientes de *gap*.

Puntos Clave del Algoritmo

- **Entrada:** Un arreglo desordenado (arr).
- **Salida:** El arreglo ordenado de menor a mayor.
- **Método:**
  - Define un valor inicial para el *gap* (normalmente la mitad del tamaño del arreglo).
  - Realiza un ordenamiento por inserción dentro de los segmentos separados por el *gap*.
  - Reduce el *gap* en cada iteración hasta que sea 1, momento en el cual se realiza una última pasada de inserción estándar.
- **Complejidad:**
  - **Peor caso:** Depende de la secuencia de *gap* utilizada, pero generalmente es $O(n^2)$.
  - **Promedio:** Entre $O(n \log^2 n)$ y $O(n^{1.5})$, dependiendo de la implementación.

Para el desarrollo de este trabajo se siguió la siguiente metodología:

- Investigación documental
- Análisis de los tiempos de ejecución de ordenamiento de los algoritmos: shellsort y quicksort
- Interpretación de resultados
- Conclusiones

2.1 **Desarrollo**

Algunos autores consideran que, para medir el rendimiento de cada algoritmo, el factor más importante es el tiempo de ejecución que utiliza para ordenar un dato (Chowdhury 2016), razón por la cual esta investigación se basa en el factor de tiempo de ejecución del ordenamiento para realizar el análisis estadístico comparativo entre los cinco algoritmos antes mencionados.

Para la experimentación en la obtención de los tiempos de ejecución de cada uno de los algoritmos se consideró:

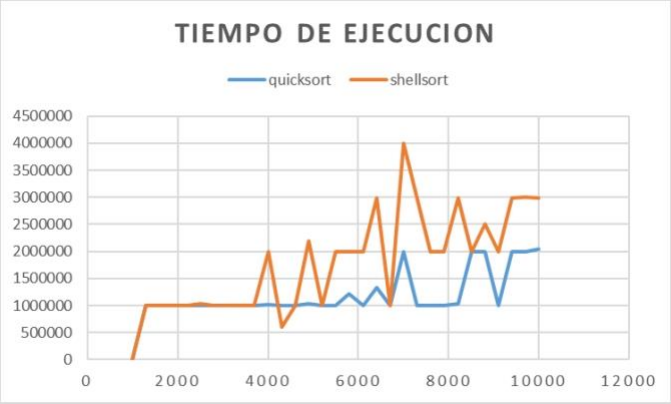
- Archivos con 1,000 a 10,000 elementos a ordenar.
- El tiempo de ejecución.
- Para cada tamaño de entrada, se ha generado instancias con los siguientes patrones: Aleatorio Uniforme (valores enteros en rango definido, se fijo semilla), ordenado ascendente y ordenado descendente.
- Se realizaron 30 ejecuciones por cada algoritmo, donde se obtuvo el siguiente resultado:

(Tabla Nro. 01)

Datos	Tiempo de ejecución en nanosegundos	
N	quicksort	shellsort
1000	0	0
1300	996600	997700
1600	997500	997800
1900	997300	997300
2200	998800	996200
2500	996900	1027400
2800	997400	997700
3100	997000	996500
3400	998300	998300
3700	998900	997100
4000	1012100	1995900
4300	993000	608200
4600	997700	997600
4900	1031000	2195700
5200	998200	997700
5500	997200	1996500
5800	1217600	1989700
6100	999700	1995300
6400	1334500	2991500
6700	996900	997300

7000	1994100	3991500
7300	997200	2993000
7600	997100	1993300
7900	998200	1994100
8200	1029700	2990200
8500	1993300	1994700
8800	1994500	2504200
9100	996100	1995300
9400	1995100	2991000
9700	1994900	3011600
10000	2044800	2991800

(Tabla Nro. 02)



- Las características de la computadora donde se ejecutaron los algoritmos fueron las siguientes:
  - Intel(R) Core (TM) i5 /10210U
  - CPU a 1.60 GHz 2.11 GHz
  - 8 GB Memoria RAM
  - Sistema operativo de 64 bits
  - Procesador x64

3 Resultados y Discusión

A partir de los datos de tu tabla, calculé la media, mediana, desviación estándar muestral, mínimo y máximo para cada algoritmo: Quicksort y Shellsort.

3.1 Resultados estadísticos

La siguiente tabla muestra los resultados estadisticos obtenidos, en base los datos de la tabla Nro. 01:

(Tabla Nro. 03)

Algoritmo	Media	Mediana	Desv. estándar	Mínimo	Máximo
Quicksort	1,180,374.19	998,200	454,179.92	0	2,044,800
Shellsort	1,781,358.06	1,993,300	942,627.17	0	3,991,500

3.2. Fórmulas utilizadas

Media:

$$\bar{x} = \frac{\sum x}{n}$$

Mediana: valor central después de ordenar los datos.

$$\mathrm{Md} = \mathrm{x}_{\frac{n}{2}+\frac{1}{2}} = \mathrm{x}_{\frac{n+1}{2}}$$

Desviación estándar muestral:

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{i=n}\left(X_i - \bar{X}\right)^2}$$

3.3 Interpretación de los resultados

Quicksort: presenta una media de aproximadamente 1.18 segundos y una desviación estándar de 454,179.92 nanosegundos.

Shellsort: presenta una media de aproximadamente 1.78 segundos y una desviación estándar de 942,627.17 nanosegundos.

Variabilidad: Shellsort tiene una mayor dispersión en sus resultados.

4. Conclusiones

4.1 Respecto del tamaño de las muestras en el rango de 1000 a 10000 en donde se emplearon los algoritmos quicksort y shellsort, dan como resultado que el algoritmo quicksort tiene el menor tiempo de ordenación en promedo, siendo mas estable en tal tendencia que el algoritmo shellsort.

Bibliografia

Adhikari, P. (2007). Review On Sorting Algorithms A comparative study on two sorting algorithms. *A Term Paper Submitted to the Faculty of Dr. Gene Boggess, Mississippi State University, In the Department of Computer Science & Engineering, Mississippi State, Mississippi*.

Alif, A. F., Islam, S. M. R., & Deb, P. (2019). *Design and implementation of sorting algorithms based on FPGA.* Paper presented at the 2019 International Conference on Computer, Communication,

Chemical, Materials and Electronic Engineering (IC4ME2).

Anderson, D. R., Sweeney, D. J., & Williams, T. A. (2008). *Estadística para administración y economía*. México: Thomson/Southwestern.

Bakieva, M., González Such, J., & Jornet, J. (2010). *SPSS: ANOVA de un factor*. Grupo de Innovación Educativa. Universitat de Valencia.

Buradagunta, S., Bodapati, J. D., Mundukur, N. B., & Salma, S. (2020). Performance Comparison of Sorting Algorithms with Random Numbers as Inputs *Ingénierie des Systèmes d'Information, 25*(1), 113-117.

Chowdhury, N. (2016). *A Java Based Tool to Monitor Execution Time of Different Sorting Algorithms.* East West University.

García, D. G., & Velázquez, J. L. A. (2019). Comparación entre algoritmos de ordenamiento paralelizados en Java. *Pistas Educativas, 41*(133).

Hildebrand, D. K., & Ott, L. (1998). *Estadística aplicada a la administración ya la economía*: Addison Wesley Logman de MÚxico, SA.

Htwe Htwe, A. (2019). Analysis and Comparative of Sorting Algorithms. *International Journal of Trend in Scientific Research and Development, 3*(5), 1049-1053.

Iskandar, I. D., Amirulloh, I., Pertiwi, M. W., Kusmira, M., Hikmah, A. B., & Supriadi, D. (2020). Analysis of bubble sort and insertion sort algorithm on memory efficiency using data mining approach.

Jurnal Pilar Nusa Mandiri, 16(1), 89-96.

Khreisat, L. (2018). A Survey of Adaptive QuickSort Algorithms. *International Journal of Computer Science and Security (IJCSS), 12*(1), 1.

Kumar, S. (2020). An Empirical Study on Algorithms-Conquer Sorting Techniques. *Available at SSRN 3568396*.

Manaseer, S., & Al Hwaitat, A. K. (2018). Measuring Parallel Performance of Sorting Algorithms Bubble Sort and Bucket Sort on IMAN. *Modern Applied Science*, *12*(10).

Nasar, A. (2019). A Mathematical Analysis of student-generated sorting algorithms. *The Mathematics Enthusiast, 16*(1), 315-330.

Prajapati, P., Bhatt, N., & Bhatt, N. (2017). Performance Comparison of Different Sorting Algorithms. *International Journal of Latest Technology in Engineering, Management & Applied Sciences, 6*.

Yang, Y., Yu, P., & Gan, Y. (2011). *Experimental study on the five sort algorithms.* Paper presented at the 2011 Second International Conference on Mechanic Automation and Control Engineering.

Zhao, L., Liu, X., & Shao, X. (2016). *Comparative Analysis of Numerical Sorting Algorithms in Java Language.* Paper presented at the 2016 6th

International Conference on Advanced Design and Manufacturing Engineering (ICADME 2016).

Páginas web consultadas:

BUILT IN Algoritmo Quicksort: una descripción general, disponible en https://builtin.com/articles/quicksort

MENTORES TECH Que es Shell Sort ?, disponible en: https://www.mentorestech.com/resource-algorithms-shell-sort.php